Job Arrays

You can make use of job arrays if you are planning to run many jobs with different inputs, or a job that iterates over many inputs and is fully independent. We tend to refer to these types of jobs as throughput jobs.

Job Arrays allow you to submit many sub-jobs and parameterize the inputs of these jobs. On this page we will refer to these sub-jobs as job array tasks, or tasks for short.

When you run a job array the scheduler will set up two environment variables for each sub-job, or task, in the array:

- `SLURM_ARRAY_TASK_ID`
- `SLURM_ARRAY_TASK_COUNT`

The first is a unique ID assigned to each task. The second is the total number of tasks. With these two numbers you have the information you need to run your tasks concurrently.

The best way to run a Job Array is so that each job array task can be assigned a range of work to do. For example, if have 1000 simulations to run each with their own input file, you want to write your code so that each job array task is assigned multiple input files. If you have 4 tasks each would be assigned 250, if you have 8 tasks they'd each be assigned 125 input files, and so on. Running in this way is more efficient for the scheduler, as it doesn't have to manage as many jobs, and it saves you on startup cost, or the time it takes for the scheduler to find resources and start running for each task.

On this page we show the basic framework of how to do this, both in a scripting language like Python and in Bash, as well as a few specific examples.

Python or Julia

If your code has a big central for loop that iterates over inputs, here is how to run your code in parallel with a Job array. If your code isn't written this way and you can re-write it so it will run in a loop over your entire set of inputs, we recommend that you do that. Python code that iterates over multiple inputs will save extra startup time overall by importing packages once per set of inputs. This may not always be convenient, and if not you can refer to the bash section below.

Examples that demonstrate this way of using a job array are available for both Python and Julia. These are also available on Engaging at the path `/orcd/examples/001/teaching-examples`.

You will need to add the following lines to take in two inputs. Make sure both `my_task_id` and `num_tasks` are in scope when you run your for loop.

Python

```
# Grab the arguments that are passed in
my_task_id = int(sys.argv[1])
num_tasks = int(sys.argv[2])
```

Julia

```
# Grab the arguments that are passed in
task_id = parse(Int,ARGS[1])
num_tasks = parse(Int,ARGS[2])
```

This grabs two arguments that we will pass into the script: a task ID and the number of tasks. Next you will take whatever you are iterating over and filter out the elements assigned to the current task (`my_task_id`):

Python

```
# Assign indices to this process/task
my_arr = arr[my_task_id:len(arr):num_tasks]
```

Here we are taking the array of inputs `arr`, extracting the elements assigned to `my_task_id` and putting them in `my_arr`. This splits up the array `arr` using a cyclic distribution based on `my_task_id` and `num_tasks`. For example, if there are 32 tasks, Task 1 will have `my_arr` 0, 32, 64, 96, ..., Task 2 will have `my_array` 1, 33, 65, 97, ..., and Task 32 will have `my_array` 31, 63, 95, and so on.

Julia

```
my_arr = arr[task_id+1:num_tasks:length(fnames)]
```

Julia Array Indexing

Julia arrays are one-based. If we start our job array indexing at 0 we need to add 1 to `task_id` as shown above.

Here we are taking the array of inputs `arr`, extracting the elements assigned to `my_task_id` and putting them in `my_arr`. This splits up the array `arr` using a cyclic distribution based on `my_task_id` and `num_tasks`. For example, if there are 32 tasks, Task 1 will have `my_arr` 1, 33, 65, 97, ..., Task 2 will have `my_array` 2, 24, 66 ..., and Task 32 will have `my_array` 32, 64, 96, and so on.

I'll then iterate over `my_arr` in the for loop instead of `arr`:

Python

```python
for element in my_arr:
    # do some work
```

Julia

```julia
for element in my_arr
    # do some work
end
```

The full script will look something like this:

Python

iterate_over_arr.py

```python
import os, sys

# Replace with your array of inputs
# This example uses numbers 0-256
arr = range(256)

# Grab the arguments that are passed in
# This is the task id and number of tasks that can be used
# to determine which indices this process/task is assigned
my_task_id = int(sys.argv[1])
num_tasks = int(sys.argv[2])

# Assign indices to this process/task
my_arr = arr[my_task_id:len(arr):num_tasks]

for num in my_arr:
    # Do something with num
    # Your code goes here
```

Julia

iterate_over_arr.jl

```
# Replace with your array of inputs
# This example uses numbers 1-256
arr = 1:256

# Grab the argument that is passed in
# This is the index into fnames for this process
task_id = parse(Int,ARGS[1])
num_tasks = parse(Int,ARGS[2])

# Check to see if the index is valid (so the program exits cleanly if the wrong
indices are passed)
for i in task_id+1:num_tasks:length(arr)

    num = arr[i]

    # Do something with num
    # Your code goes here

end
```

To run this with a Job Array with 4 tasks I would use the following job script:

Python

my_job_array.sh

```
#!/bin/bash

#SBATCH -p mit_normal
#SBATCH -o myjob.log-%A-%a
#SBATCH -a 0-3

# Load Anaconda Module
module load miniforge

echo "My SLURM_ARRAY_TASK_ID: " $SLURM_ARRAY_TASK_ID
echo "Number of Tasks: " $SLURM_ARRAY_TASK_COUNT

python iterate_over_arr.py $SLURM_ARRAY_TASK_ID $SLURM_ARRAY_TASK_COUNT
```

Julia

my_job_array.sh

```
#!/bin/bash

#SBATCH -p mit_normal
#SBATCH -o myjob.log-%A-%a
```

```
#SBATCH -a 0-3

# Load Anaconda Module
module load julia

echo "My SLURM_ARRAY_TASK_ID: " $SLURM_ARRAY_TASK_ID
echo "Number of Tasks: " $SLURM_ARRAY_TASK_COUNT

julia iterate_over_arr.jl $SLURM_ARRAY_TASK_ID $SLURM_ARRAY_TASK_COUNT
```

The first job flag (`-o myjob.log-%A-%a`) specifies the output file name, which will be appended with the Array Job ID (`%A`) and Task ID (`%a`). The second flag `-a 0-3` requests a job array with array task indices 0, 1, 2, 3. Here we specify zero-based indices because Python arrays are zero-based. For a one-based language like Matlab/Octave or Julia, we would use indices `1-4` instead.

As mentioned earlier, `$SLURM_ARRAY_TASK_ID` is a unique ID assigned to each task and `$SLURM_ARRAY_TASK_COUNT` is the total number of tasks. In the last line of the script we run the python script `iterate_over_arr.py` and pass both environment variables into the script.

The last step is to run the job with `sbatch` :

```
sbatch my_array_job.sh
```

When you run `squeue --me` you will see which job array tasks are running and which are still pending. Each running job array task will be on its own line, as shown below. Pending tasks will be listed on a single line together. Note the Job IDs have two numbers. The first number is the Job Array ID, a Job ID given to the entire array, the second is the Task ID.

```
        JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
62445052_0 mit_norma my_job_a username  R       0:05      1 node2704
62445052_1 mit_norma my_job_a username  R       0:05      1 node2704
62445052_2 mit_norma my_job_a username  R       0:05      1 node2704
62445052_3 mit_norma my_job_a username  R       0:05      1 node2704
```

Bash

If you can't re-write your code as described in the Python or Julia example above, you can accomplish the same thing in your job script using `bash` . I will start with the basic framework and then give some examples of some common variations.

For simplicity, let's say we have an application `my_cmd` that takes a number as an input. To run this on a single number we'd start with a job script that looks like this:

run_my_cmd_serial.sh

```
#!/bin/bash

#SBATCH -p mit_normal
#SBATCH -o my_cmd_serial.log-%j

# Set the number to run my_cmd on
export MY_NUM=1

my_cmd $MY_NUM
```

Few Inputs

If you plan to run this on relatively few numbers, say less than around 100, and `my_cmd` runs for longer than a few seconds you can use something like this script below. Let's say we want to run `my_cmd` on numbers 1-32. We can create a job array with 32 tasks each assigned an index 1-32 by adding the flag `-a 1-32`. Here is the example script

few_inputs.sh

```
#!/bin/bash

#SBATCH -p mit_normal
#SBATCH -o my_cmd_array32.log-%A-%a
#SBATCH -a 1-32

# Set the number to run my_cmd on
export MY_NUM=$SLURM_ARRAY_TASK_ID

my_cmd $MY_NUM
```

Notice we are setting `$MY_NUM` to `$SLURM_ARRAY_TASK_ID` and passing it into `my_cmd`. I've also changed the output job flag (`-o my_cmd_array32.log-%A-%a`) so it will be appended with the Array Job ID (`%A`) and Task ID (`%a`) to the name of the log file.

The last step is to run the job with `sbatch`:

```
sbatch few_inputs.sh
```

When you run `squeue --me` you will see which job array tasks are running and which are still pending. Each running job array task will be on its own line, as shown below. Pending tasks will be listed on a single line together. Note the Job IDs have two numbers. The first number is the Job Array ID, a Job ID given to the entire array, the second is the Task ID.

```
        JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
62445052_0 mit_norma my_job_a username  R       0:05      1 node2704
62445052_1 mit_norma my_job_a username  R       0:05      1 node2704
62445052_2 mit_norma my_job_a username  R       0:05      1 node2704
62445052_3 mit_norma my_job_a username  R       0:05      1 node2704
...
62445052_32 mit_norma my_job_a username  R       0:05      1 node2704
```

Many Inputs

The script above works well for smaller numbers of tasks, but doesn't scale well to larger numbers. Because there are limited resources, you end up spending more time waiting for available resources than you do running your application. The scheduler will also slow down when it has to manage very large numbers of jobs, so we limit the number of jobs each user can run on each partition. Here is an approach that allows you to run many inputs with fewer job array tasks.

Let's say we want to run `my_cmd` on the numbers 1-256. To run this as a job array with 32 tasks, for example, we can use the following job script:

many_inputs.sh

```
#!/bin/bash

# Scheduler Options
#SBATCH -p mit_normal
#SBATCH -o myout.log-%A-%a
#SBATCH -a 0-31

echo "My SLURM_ARRAY_TASK_ID: " $SLURM_ARRAY_TASK_ID
echo "Number of Tasks: " $SLURM_ARRAY_TASK_COUNT

export MAX_NUM=256

my_array=( $(seq $SLURM_ARRAY_TASK_ID $SLURM_ARRAY_TASK_COUNT $MAX_NUM) )

# Iterate over my_array
for IDX in "${my_array[@]}"; do
    my_cmd $IDX
done
```

Bash Array Indexing

Bash arrays are zero-based. To make indexing easier start your job array indices at 0.

As mentioned earlier, `$SLURM_ARRAY_TASK_ID` is a unique ID assigned to each task and `$SLURM_ARRAY_TASK_COUNT` is the total number of tasks.

In line 13:

```
13  my_array=( $(seq $SLURM_ARRAY_TASK_ID $SLURM_ARRAY_TASK_COUNT $MAX_NUM) )
```

We are creating an array of the numbers we want the current task to pass into `my_cmd`. This splits up the numbers 1-256 using a cyclic distribution. Since there are 32 tasks, Task 1 will have `my_array` 1, 33, 65, 97, ..., Task 2 will have `my_array` 2, 24, 66 ..., and Task 32 will have `my_array` 32, 64, 96, and so on.

In the final few lines we iterate over `my_array` with a for loop and run `my_cmd` on each number in `my_array` in turn:

```
15  # Iterate over my_array
16  for IDX in "${my_array[@]}"; do
17      my_cmd $IDX
18  done
```

The last step is to run the job with `sbatch`:

```
sbatch many_inputs.sh
```

When you run `squeue --me` you will see which job array tasks are running and which are still pending. Each running job array task will be on its own line, as shown below. Pending tasks will be listed on a single line together. Note the Job IDs have two numbers. The first number is the Job Array ID, a Job ID given to the entire array, the second is the Task ID.

```
         JOBID PARTITION     NAME     USER ST       TIME  NODES NODELIST(REASON)
62445052_0 mit_norma my_job_a username  R       0:05      1 node2704
62445052_1 mit_norma my_job_a username  R       0:05      1 node2704
62445052_2 mit_norma my_job_a username  R       0:05      1 node2704
62445052_3 mit_norma my_job_a username  R       0:05      1 node2704
...
62445052_32 mit_norma my_job_a username  R       0:05      1 node2704
```

Even though we have 256 numbers we are iterating through, we have 32 job array tasks running. Each job array task will be assigned 8 numbers in `my_array` (256/32 = 8).

This is a very minimal example. In many cases you'll need a bit more than this to run your job array. In Job Array Models we show what adjustments to make for some common situations: using inputs from a file and using files as inputs. We recommend using these example scripts below as models for your own job array jobs. Copy the script and make updates as needed.

Job Array Models

Inputs from a File

In this case you will need a plain text file where each line contains one input. This will also work if your code takes multiple command line arguments, list them separated by a space the same way you would at the command line.

Here is the example script:

inputs_from_file.sh

```
#!/bin/bash

# Scheduler Options
#SBATCH -p mit_normal
#SBATCH -o myout.log-%A-%a
#SBATCH -a 1-4

echo "My SLURM_ARRAY_TASK_ID: " $SLURM_ARRAY_TASK_ID
echo "Number of Tasks: " $SLURM_ARRAY_TASK_COUNT

# Specify Input File
INPUT_FILE=file.txt
NUM_LINES="$(wc -l < $INPUT_FILE)"

# Distribute line numbers
MY_LINE_NUMS=( $(seq $SLURM_ARRAY_TASK_ID $SLURM_ARRAY_TASK_COUNT $NUM_LINES)
)

# Iterate over $MY_LINE_NUMS
for LINE_IDX in "${MY_LINE_NUMS[@]}"; do

    # Get the $LINE_IDX-th line from $INPUT_FILE
    INPUT="$(sed "${LINE_IDX}q;d" $INPUT_FILE)"

    # Run my_cmd on $INPUT
    my_cmd $INPUT
done
```

sed Command Indexing

The `sed` command, which we use to retrieve lines from the input file, is one based. To make indexing easier start your job array indices at 1.

Use this Script

To use this script specify the name of your input file in line 12 and adjust line 25 to run your application.

This example works very similarly to the one above, with a few additions. Lines 12 specifies the name of the file containing the input strings, and line 13 finds the number of lines in the file using the `wc` or "word count" command.

```
11  # Specify Input File
12  INPUT_FILE=file.txt
13  NUM_LINES="$(wc -l < $INPUT_FILE)"
```

Line 22 within loop above uses the `sed` command to extract the current iteration's line from the file. The `sed` (Stream Editor) command can be used for many things, including extracting parts of a file as well as replacing or deleting text from a file. The `sed` command uses one-based indexing (it starts counting at 1 instead of 0), so it is easiest to start job array indices at 1 (see line 6 above).

```
21  # Get the $LINE_IDX-th line from $INPUT_FILE
22  INPUT="$(sed "${LINE_IDX}q;d" $INPUT_FILE)"
```

Files as Inputs

This example shows how to pass in a directory of files as inputs.

inputs_from_file.sh

```
1   #!/bin/bash
2
3   # Scheduler Options
4   #SBATCH -p mit_normal
5   #SBATCH -o myout.log-%A-%a
6   #SBATCH -a 0-3
7
8   echo "My SLURM_ARRAY_TASK_ID: " $SLURM_ARRAY_TASK_ID
9   echo "Number of Tasks: " $SLURM_ARRAY_TASK_COUNT
10
11  # Specify Input Directory
12  INPUT_DIR=inputs/*
13  FILES=(${INPUT_DIR})
14  NUM_FILES=${#FILES[@]}
15
16  # Distribute files
17
```

```
18  MY_FILE_NUMS=( $(seq $SLURM_ARRAY_TASK_ID $SLURM_ARRAY_TASK_COUNT
19  "$(($NUM_FILES-1))") )
20
21  # Iterate over $MY_FILE_NUMS
22  for IDX in "${MY_FILE_NUMS[@]}"; do
23
24      # Get the $IDX-th file from $FILES
25      INPUT_FILE=${FILES[$IDX]}
26
27      # Run my_cmd on $INPUT_FILE
        my_cmd $INPUT_FILE
    done
```

Use this Script

To use this script specify specify the files that you are using for inputs (be sure to include a wildcard *) in line 12 and adjust line 25 to run your application.

In this example a list of files are passed into the `my_cmd` application. Line 12 specifies which files are the input files. The expansion that happens in line 13 to get the names of the files will only work if you include a wildcard in line 12. To test if you have it right run `ls $INPUT_DIR`. You should see all the files you expect to pass into your script.

```
11  # Specify Input Directory
12  INPUT_DIR=inputs/*
13  FILES=(${INPUT_DIR})
```

Line 22 gets the file assigned to the current iteration, and line 25 passes that into `my_cmd`

```
21  # Get the $IDX-th line from $INPUT_FILE
22  INPUT_FILE=${FILES[$IDX]}
23
24  # Run my_cmd on $INPUT_FILE
25  my_cmd $INPUT_FILE
```

February 19, 2025